

Secure CLI Chat Application with End-to-End Encryption

Introduction:

In today's digital world, secure communication is a critical requirement due to increasing cyber threats such as eavesdropping, man-in-the-middle (MITM) attacks, and data tampering. This project focuses on developing a Secure Command-Line Interface (CLI) Chat Application that ensures confidentiality, integrity, and authenticity of messages using modern cryptographic techniques. The application enables multiple users to communicate securely over a network using end-to-end encryption (E2EE). It is designed as a lightweight and efficient solution suitable for learning and demonstrating practical cybersecurity concepts.

Objectives:

The primary objectives of this project are:

- To implement secure real-time communication between users
- To ensure data confidentiality using encryption techniques
- To verify message integrity and authenticity using digital signatures
- To simulate secure key exchange between users
- To provide a simple and user-friendly CLI-based interface

Technologies Used:

- **Programming Language:** Python
- **Framework:** Flask-SocketIO
- **Networking:** Socket.IO (WebSockets)
- **Cryptography Libraries:** cryptography / custom utilities
- **Concurrency:** Threading

System Architecture:

The system follows a client-server architecture:

- **Server:** Handles user connections, message routing, and key exchange
- **Client:** Performs encryption, decryption, signing, and verification

Workflow:

1. User connects to the server

2. Public keys are exchanged
3. Messages are encrypted using AES
4. AES keys are securely shared using RSA
5. Messages are digitally signed
6. Receiver decrypts and verifies signature

Key Features:

End-to-End Encryption (E2EE)

Messages are encrypted using AES symmetric encryption. The AES key is securely exchanged using RSA asymmetric encryption, ensuring that only the intended recipient can read the message.

Digital Signature

Each message is digitally signed using the sender's private key. The receiver verifies the signature using the sender's public key, ensuring message integrity and authenticity.

Key Exchange Mechanism

Each user generates a public-private key pair. Public keys are shared through the server and used for encryption and verification.

Online User Management

The application maintains a list of active users and allows secure communication between them.

Message Queueing

If a user's public key is not available, messages are temporarily stored and automatically sent once the key is received.

Multi-threaded Input Handling

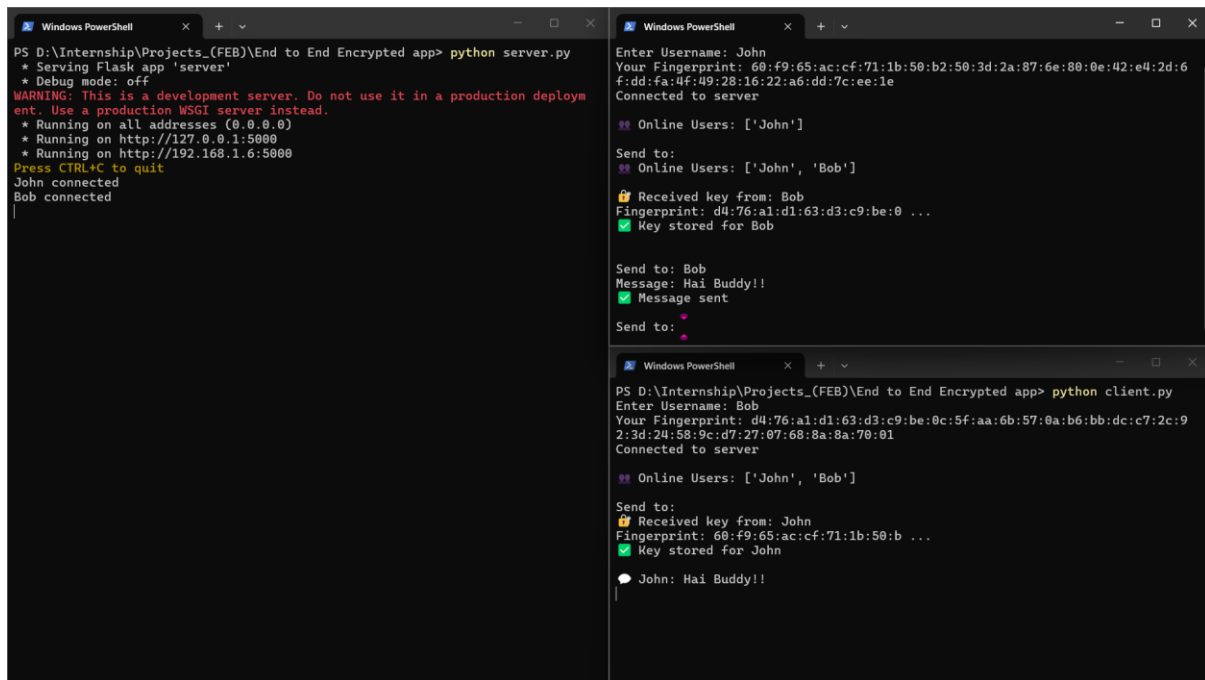
The client uses threading to handle user input and incoming messages simultaneously.

Security Features:

- Encryption prevents unauthorized access
- Digital signatures prevent message tampering
- Fingerprint generation allows key verification
- Protection against unauthorized message injection

Results:

The application successfully enables secure real-time communication between multiple users. Messages are encrypted, signed, and verified correctly. The system performs reliably under normal usage and demonstrates core cybersecurity principles effectively.



```
PS D:\Internship\Projects_(FEB)\End to End Encrypted app> python server.py
* Serving Flask app 'server'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://192.168.1.6:5000
Press CTRL+C to quit
John connected
Bob connected

Enter Username: John
Your Fingerprint: 60:f9:65:ac:cf:71:1b:50:b2:50:3d:2a:87:6e:80:0e:42:e4:2d:6f:dd:fa:4f:49:28:16:22:a6:dd:7c:ee:1e
Connected to server

Online Users: ['John']

Send to:
Online Users: ['John', 'Bob']

Received key from: Bob
Fingerprint: d4:76:a1:d1:63:d3:c9:be:0 ...
Key stored for Bob

Send to: Bob
Message: Hai Buddy!!
Message sent

Send to:

PS D:\Internship\Projects_(FEB)\End to End Encrypted app> python client.py
Enter Username: Bob
Your Fingerprint: d4:76:a1:d1:63:d3:c9:be:0c:5f:aa:6b:57:0a:b6:bb:dc:c7:2c:92:3d:24:58:9c:d7:27:07:68:8a:8a:70:01
Connected to server

Online Users: ['John', 'Bob']

Send to:
Received key from: John
Fingerprint: 60:f9:65:ac:cf:71:1b:50:b ...
Key stored for John

John: Hai Buddy!!
```

Conclusion:

This project demonstrates the practical implementation of secure communication using cryptographic techniques. It provides a strong foundation in concepts such as encryption, digital signatures, and secure key exchange. The application can be further extended with advanced features like attack detection, logging, and graphical interfaces.